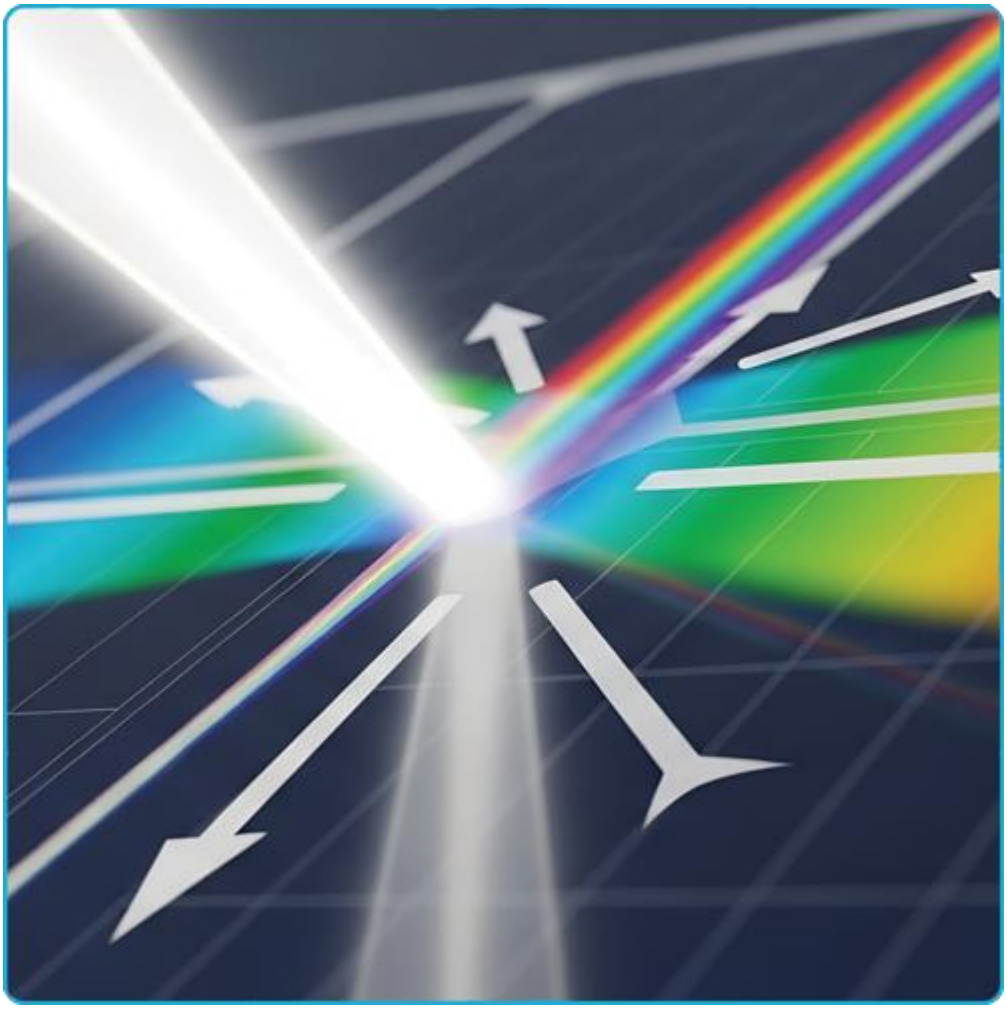
Produktrapport



Deltagere: Freja Pedersen, Kasper Korthsen og Silas Aagaard

Projekttitel: Måling af lys

Startdato: 19. marts, 2026

Slutdato: 17. april, 2026

Skole: Mercantec, Viborg

Vejleder: Mathias Steenberg

Indholdsfortegnelse

Kravspecifikation 2
Projektbeskrivelse (Freja) 3
Tech Stack (tekst af Kasper, diagram af Silas) 3
Domænemodel (Silas) 5
Krav 6
Funktionelle krav (Freja) 7
Ikke-funktionelle krav (Freja og Kasper) 7
Afgrænsning (Freja) 7
Vejledning 8
Twingate-netværk (Freja) 9
Adgang (Freja) 9
Arduino (Kasper) 9
Hjemmeside (Freja) 9
Forside (Freja) 9
Loginside (Freja) 10
Dashboard (Freja) 11
Teknisk dokumentation 15
Datakilde 16
Arduino (Kasper) 16
Back End 17
Teknologivalg 17
Docker (Kasper) 17
Twingate (Freja) 19
Python (Kasper) 19
Psycopg2 (Kasper) 21
Pika (Kasper) 21
RabbitMQ (Kasper) 21
RestAPI (Silas) 23
Overblik (Freja) 27
Server og adgang (Freja) 27

Data og håndtering (Kasper) 28
Dataadgang og grænseflader (Silas) 28
Front End..... 31
Læsevejledning (Freja) 32
Teknologivalg (Freja) 32
Chart.js (Freja) 32
Overblik (Freja) 32
Login (Freja) 33
CSS Animation (Freja) 34
Dashboard (Freja)..... 35
JSON Generator (Freja)..... 37

Kravspecifikation

Projektbeskrivelse (Freja)

Dette projekt er til en platform, som skal hjælpe private eller firmaer med at finde mere optimale steder at opstille solceller/solfangere. Måleenheder placeres på ønskede steder, og sender derefter målinger på lysstyrke tilbage derfra, som kan ses og sammenlignes via en tilknyttet webapplikation.

Som en del af projektets rammer, sigter vi efter, at:

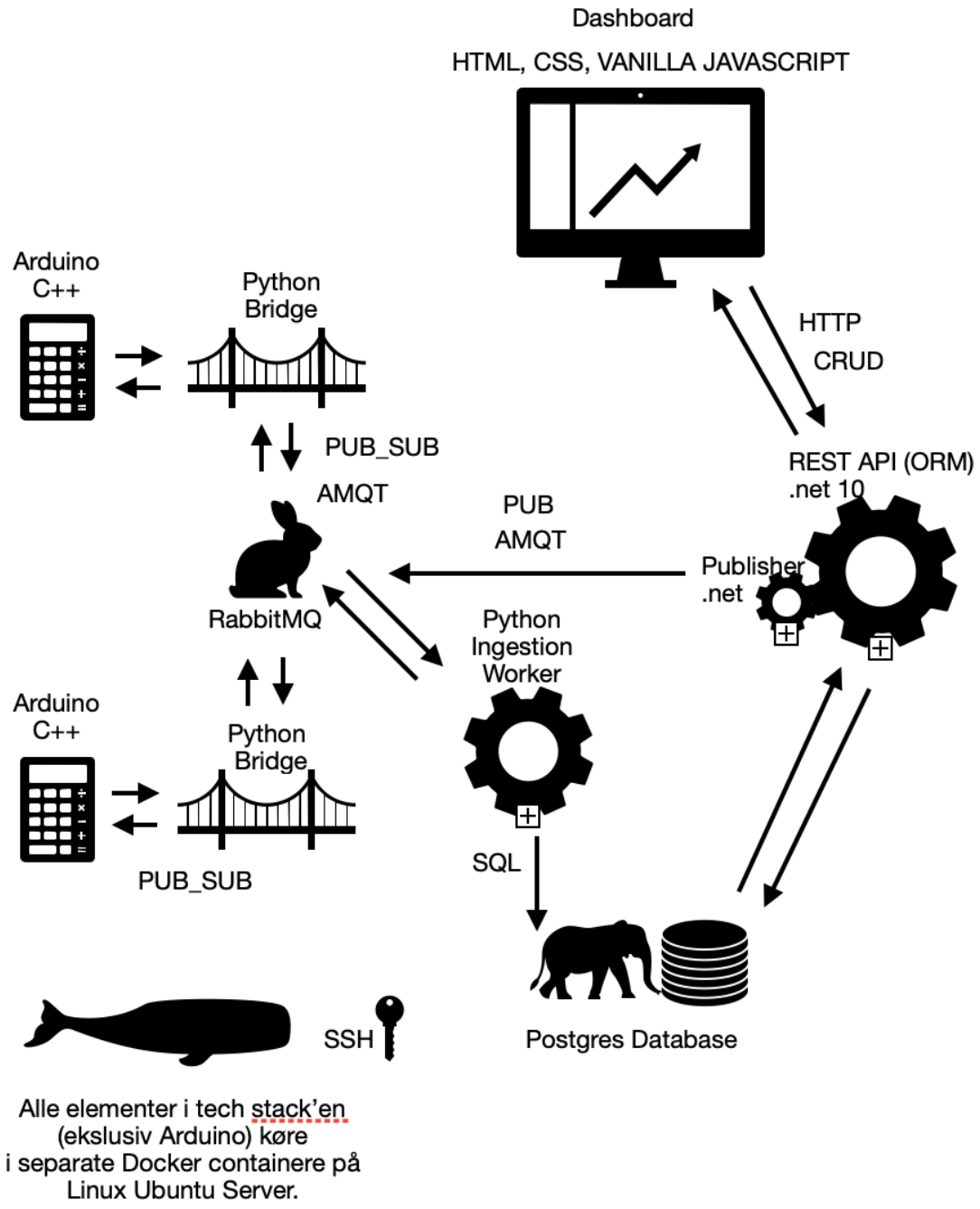
- Der skal gøres brug af måleenheder via mikrocontroller.
- Vi skal kunne tage imod målinger fra mikrocontrolleren.
- Målingerne skal indholde tidspunkt og lysintensitet.
- Datamålinger skal opbevares i en skalérbar database.
- Data skal sendes via MQTT og ikke HTTP.
- Vi skal have en API til at kører på serveren.
- APIen skal kunne hente alt data og hente for den specifikke bruger og hente seneste måling.
- Databasen skal være PostgreSQL (Postgres).
- Databasen skal være på en Linux server
- Serveren skal have rollefordeling for brugere, dvs fx ingen global root-adgang
- Der skal optimeres mod stabilitet/oppetid.
- Vores server/API skal kunne opbevares og hostes som en Docker-container

Derudover vil vi prøve at opnå dette med vores produkt:

- Der skal være data for lysintensitet og –farve
- Denne data skal visualiseres og vises via en webapplikation med et dashboard
- Man skal som bruger kunne logge ind og se sin egen data
- Visualisering af målinger i realtid
- Målinger kan startes af brugeren i dashboardet
- Sørge for at der er et DeviceID på din installation, så hvis man har flere installationer kan skelne mellem dem.
- Gøre det det muligt at have flere installationer ad gangen.

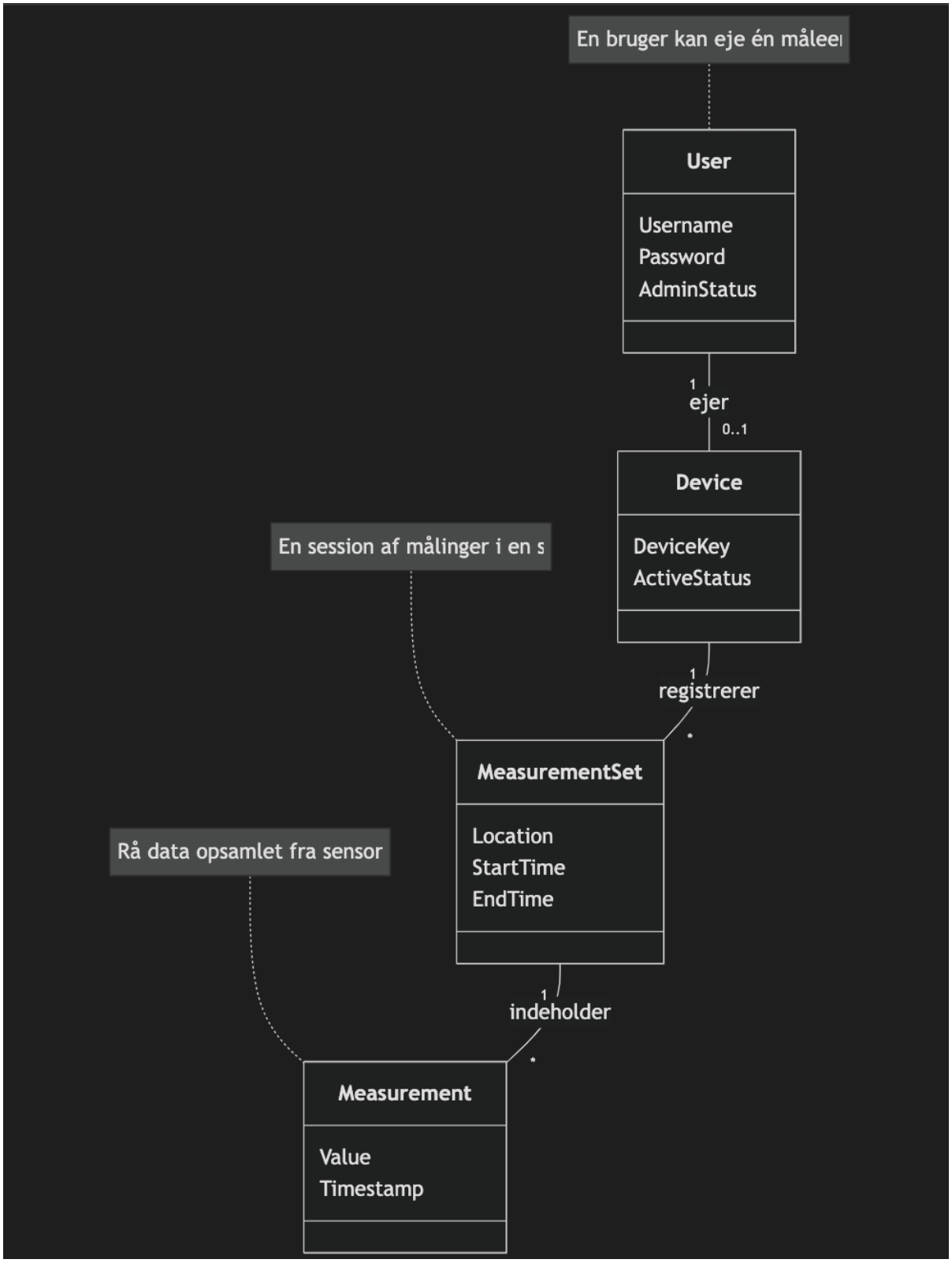
Tech Stack (tekst af Kasper, diagram af Silas)

Tech stacken viser at vi modtager dataen fra vores arduino (lys intensitet og tid) som vi så bruger en Python producer til at dekode og sende videre til vores RabbitMQ som er vores messagebroker som så bruger vores Python consumer til at store dataen i vores Postgres Database som så bliver hentet af vores API som så kan hente dataen, som gør at vi kan hente dataen på vores frontend2



Domænemodel (Silas)

Herunder ses projektets domænemodel, som visualiserer forholdet mellem vores POCO-modeller (Plain Old CLR Object), database-context og repositories på et mere konceptuelt niveau.



Krav

Funktionelle krav (Freja)

Til vores projekt har vi følgende krav, udtrykt som brugerhistorier/user stories:

- Som bruger vil jeg gerne have min egen profil, så jeg kan se min egen data
- Som bruger vil jeg gerne være i stand til at se data fra et vilkårligt tidspunkt
- Som bruger vil jeg gerne se min data, som det kommer ind
- Som bruger vil jeg gerne se min data på en præsentabel måde, som nemt giver overblik
- Som bruger vil jeg gerne have mulighed for at vælge, hvor meget data jeg ser ad gangen
- Som bruger vil jeg gerne have en installationsguide

Derudover har vi nogle punkter, som vi gerne vil se, om vi kan nå:

- Som bruger skal min profil have sikkerhed, så kun jeg kan se og tilgå mine ting
- Som bruger vil jeg gerne kunne aktivere og deaktivere mine måleenheder via fjernstyring
- Som bruger vil jeg gerne være i stand til at give mine måleenheder navne, så jeg ved, hvilket data er hvilket
- Som bruger vil jeg gerne kunne give mig selv en adgangskode
- Som bruger vil jeg gerne være i stand til at ændre min adgangskode
- Som bruger vil jeg gerne have ID på mine enheder, så jeg ved, hvilken er hvilken

Ikke-funktionelle krav (Freja og Kasper)

- Serveren skal have så stabil og lang oppetid som muligt
- Vores database skal bruge Postgres
- Serveren skal køre Linux
- API'en skal køre i en docker container
- Udviklerne skal ikke benytte root-adgang på serveren
- Brugeren må kun tilgå sin egen data
- Målingerne skal visualiseres med et diagram og en tabel

Afgrænsning (Freja)

På grund af tidsbegrænsninger, bliver der ikke et fokus på brugerkonti og opdeling af data for brugere. Derudover, pga. vores udstyr, måler vi kun lysstyrke, og ikke andre detaljer såsom lysfarve.

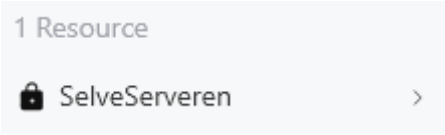
Vejledning

Twingate-netværk (Freja)

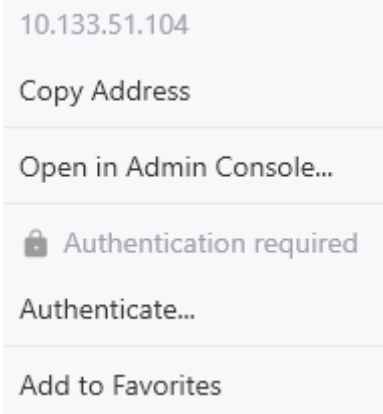
Adgang (Freja)

Når brugeren er tilknyttet det private netværk, skal de stadig logge ind for at forbinde til serveren. Når Twingate kører, kan et lille ikon findes i værktøjslinjen.

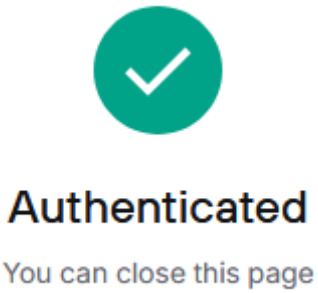
Når det klikkes på, kan brugeren se sin bruger, det tilknyttede netværk, og netværkets resurser – herunder serveren.



Klikker brugeren på serveren, kommer endnu en menu frem med nogle muligheder, hvoraf de skal klikke på "Authenticate..."



Derefter åbner den en fane i brugerens webbrowser, hvor de skal bruge deres Twingate loginmetode.
Efter det er gjort korrekt, viser siden et flueben, og brugeren kan frit lukke for browserfanen.
Derefter er der adgang til serveren i noget tid, eller indtil brugeren genstarter Twingate-klienten.



Arduino (Kasper)

Kunden tilslutter sin Arduino til et USB i computeren og stiller et sted kunden vil måle sollyset igennem dagen for at kunne finde en optimal plads til at placere solceller, denne information vil så blive videre sendt gennem og fremvist på vores hjemmeside som kunden kan tilgå.

Hjemmeside (Freja)

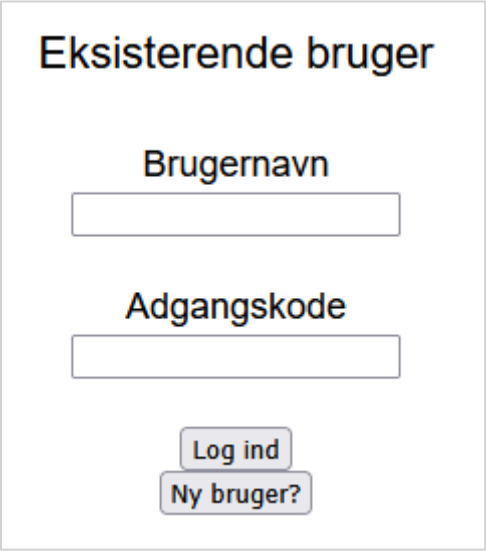
Hjemmesiden har 3 sider: en forside, en side til login, og en side til dashboard.

Forside (Freja)

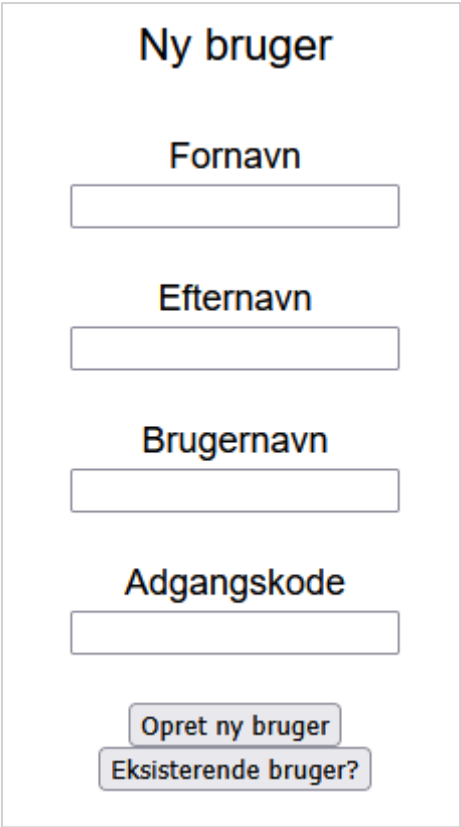
Forsiden er nuværende tom, bortset fra en delt bar i toppen, som alle siderne deler. For at brugeren kan komme videre til loginsiden, skal de trykke på knappen i øverste højre hjørne, som siger “Log ind”.

Loginside (Freja)

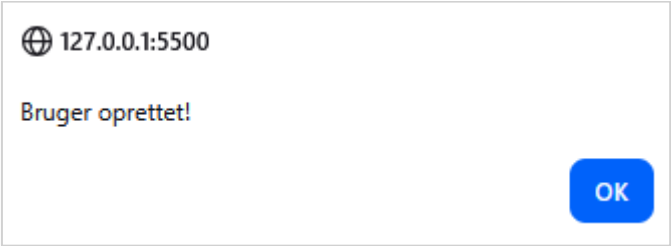
På loginsiden findes der to formularer, én til at logge ind, og én til at oprette en ny bruger.



Hvis brugeren derimod ikke har en bruger(konto), skal de trykke på knappen ”Ny bruger?”, som bytter på de to formularer. Derefter kommer en ny formular frem.

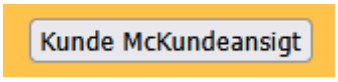


Her skal brugeren indtaste deres fornavn og efternavn, og derefter et ønsket brugernavn og adgangskode til at logge ind i fremtiden. Når det er gjort, sender siden et lille popup som feedback.



Når bruger(konto)en er oprettet, kan brugeren logge ind med de detaljer, som de oprettede sig med. Det fører dem videre til deres dashboard.

Mens brugeren er logget ind, er deres navn synligt i stedet for "Log ind" knappen i sidens topbar.



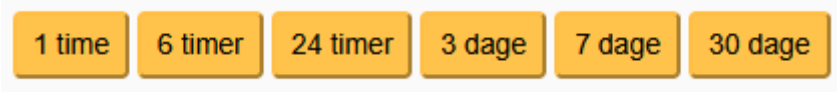
Dashboard (Freja)

Når brugeren tilgår siden, vil de blive mødt med alle de forskellige dele af deres dashboard.

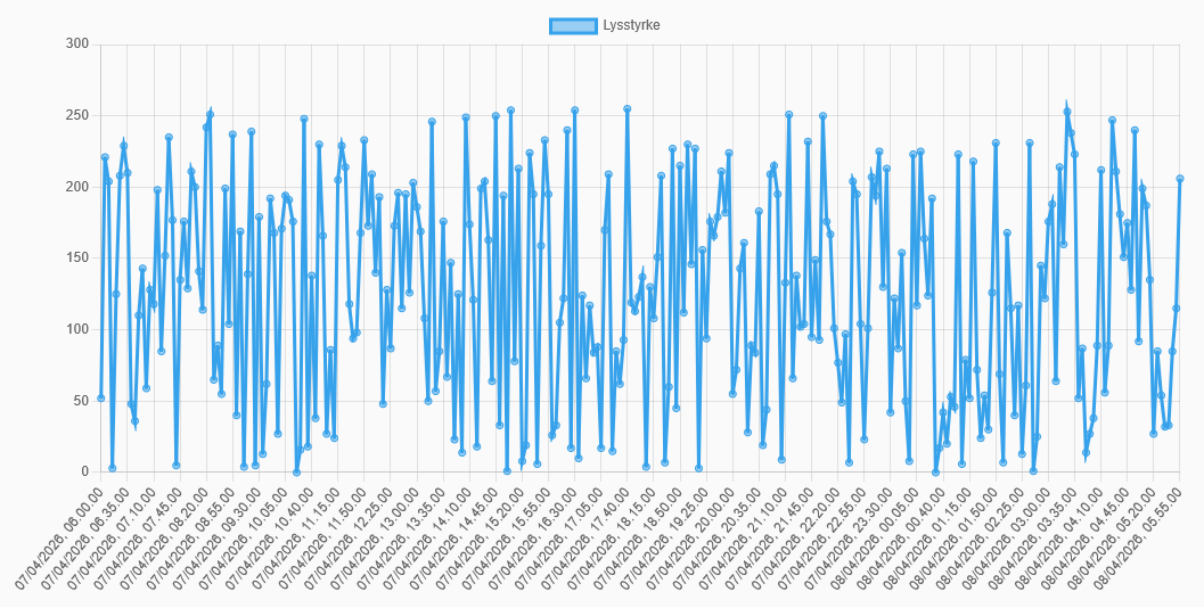
For at ændre i det tidspunkt, brugeren gerne vil se data fra, skal de ændre i felterne øverst til højre.

Til venstre vises der en dato. Den kan ændres ved at klikke på kalenderikonet, hvorefter en kalender vises, og den ønskede dato kan vælges med et klik. Til højre for datoen findes der to felter, som repræsenterer et klokkeslæt. Det venstre felt er timer, og det højre minutter. Disse to felter kan brugeren selv skrive i for at få et ønsket tidspunkt. Knappen "Nu" indstiller dato og tid til computerens nuværende tid.

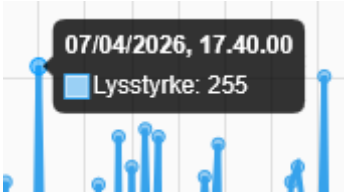
For at vælge et ønsket interval, som der skal vises data fra, skal brugeren trykke på en af knapperne her.



Når brugeren trykker på en af knapperne, hentes data i det valgte interval fra den indtastede dato og tid, hvis det findes. For at undgå at overvælde brugeren med data, er der større mellemrum mellem de forskellige datapunkter ved større tidsintervaller. Disse datapunkter vises derefter i dashboardets diagram og tabel, hvor det kan ses og undersøges.



Kunden kan se en visuel repræsentation af deres data via det store kurvediagram øverst på siden, hvor tid er markeret på x-aksen, og lysstyrke på y-aksen. Hvis kunden gerne vil se på et datapunkt i detaljer, kan de føre musemarkøren hen over det.



Tid	Lysintensitet
07/04/2026	
06.00.00	52
06.05.00	221
06.10.00	204
06.15.00	3
06.20.00	125
06.25.00	208
06.30.00	229
06.35.00	210
06.40.00	48
06.45.00	36
06.50.00	110
06.55.00	143
07.00.00	59
07.05.00	128
07.10.00	118
07.15.00	198
07.20.00	85
07.25.00	152
07.30.00	235
07.35.00	177
07.40.00	5
07.45.00	135
07.50.00	176
07.55.00	129
08.00.00	211

Kunden kan også se sin data i en tabel under diagrammet. Her vises hvert individuelt datapunkt, og tabellen kan der bladres (eller "scrolles") i for at se nyere eller ældre data i den. Den dato, som tidspunkterne tilhører, vises øverst i tabellen, og er altid synlig. Hvis den data, der vises, strækker sig over flere dage, bliver de vist, som når dem, og den synlige dato bliver udskiftet, når der ikke er ældre data synlig.

07/04/2026	
22.30.00	150
23.00.00	239
23.30.00	203
08/04/2026	
00.00.00	49
00.30.00	129
01.00.00	199

Teknisk dokumentation

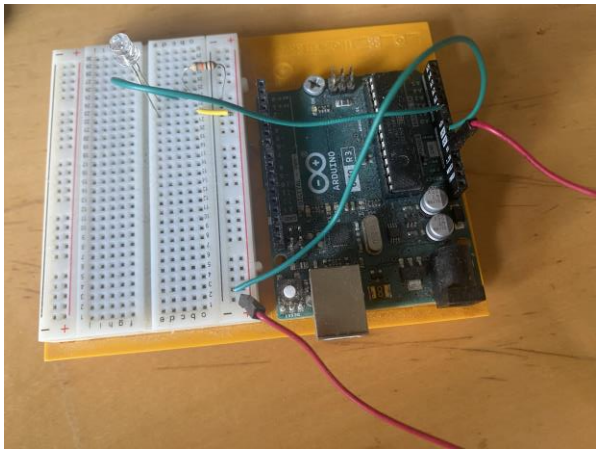
Datakilde

Arduino (Kasper)

Vi har valgt at benytte os af en phototransistor tilsluttet en arduino for at kunne producere en stor mængde data, ved at stille den et sted for at måle belysningen i området for at man ville kunne få en bedre indsigt i hvor man bedst ville kunne placere sine solceller for at få størst udbytte, dette er blevet gjort med en arduino uno r3.

```
int sensorPin = A0;     // Pin forbundet til fototransistoren
int sensorValue = 0;    // Variabel til at gemme værdien

void setup() {
  Serial.begin(9600);  // Start seriel kommunikation
}

void loop() {
  sensorValue = analogRead(sensorPin); // Læs værdien (0-1023)

  Serial.println(sensorValue);

  delay(15000); // Vent 15 sekunder før næste måling
}
```

/ Billedet oven over viser den Arduino/C++ kode der er sat op til at blive uploadet til arduinoen først forbinder vi den til pin A0 som phototransistoren er sluttet til gennem breadboard og til pinnen på Arduinoen, vi starter den ud med en variabel hvor starten er 0 så den kan tage tal som er den data der bliver gemt, så starter vi den til at kører på baudraten 9600 som giver tilladelse til at sende data over til pc'en, og her har vi så vores loop som kører når vi starter den og til at vi slukker den hvor den så giver os en måling på hvor meget spænding phototransistoren sender ud som ligger mellem 0-1023 og det printer vi så ud som en tekstlinje så Python-scriptet kan læse det og formatere det til JSON og det har vi så sat til at den skal tage en måling hvert 15ende sekund. /



/ Billedet oven over viser vores opsætning af Arduinoen hvor man kan se at den er sluttet til sensor pin a0 /

Back End

Teknologivalg

Docker (Kasper)

Docker er valgt for at vi nemt har kunne køre filer med blandt andet docker-compose så vi ikke ville kunne ramme ind i pladsmangel på serveren, dette er gjort ved hjælp af nano til at skrive filerne ind i de korrekte mapper og gøre brug af services til at køre f.eks vores rabbitMQ og vores API.

```
es > docker-compose.yml
services:
  postgres:
    image: postgres:15
    container_name: postgres
    environment:
      POSTGRES_USER: kasper
      POSTGRES_PASSWORD: H5kode
      POSTGRES_DB: sensor
    ports:
      - "5432:5432"
    volumes:
      - ./data:/var/lib/postgresql/data
    restart: always
```

/ Billedet oven over viser hvor vi opretter en container som kører servicen postgres, derefter bruger vi image postgres:15 da det er standard for docker, så har vi givet den container navnet postgres og så har vi vores enviroment data med brugernavn, kode og hvilken db den skal ramme “sensor”, derefter ligger vi den til at lytte på port 5432 da det er standarden som den kører med så vi kan tilgå den på localhost:5432, og så har vi volumes som er der hvor vi retter vores data hen så den ligger i den mappe der er

udpeget, så kører vi restart: always hvilket gør at den starter automatisk hvis enten docker crasher eller vi starter docker op fra ny. /

```
nq > docker-compose.yml
services:
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    ports:
      - "5672:5672"
      - "15672:15672"
    restart: always
```

/ Billedet oven over viser igen den container vi kører og hvilken service og det image vi kører af rabbitMQ og så hvad navn vi har givet den, hvilke porte den lytter på så vi kan tilgå den på localhost:15672 og at den også kører vores restart så vi sikrer os at den altid er oppe og køre /

```
# Stage 1: Base image til runtime
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS base
USER $APP_UID
WORKDIR /app
EXPOSE 8080
EXPOSE 8081

# Stage 2: Build image med SDK
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
ARG BUILD_CONFIGURATION=Release
WORKDIR /project
COPY ["project/API/h5_2o_MAIN.csproj", "API/"]
RUN dotnet restore "./API/h5_2o_MAIN.csproj"
COPY project/API/. "./API/"
WORKDIR "/project/API"
RUN dotnet build "h5_2o_MAIN.csproj" -c $BUILD_CONFIGURATION -o /app/build

# Stage 3: Publish
FROM build AS publish
ARG BUILD_CONFIGURATION=Release
RUN dotnet publish "h5_2o_MAIN.csproj" -c $BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false

# Stage 4: Final image (kun runtime)
FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "h5_2o_MAIN.dll"]
```

/ Billedet oven over viser en multistage dockerfile, hvor at i stage 1 der starter vi med at kører et runtime image da vi ikke kan bygge .net i docker men kun køre, derefter bruger vi $APP_UID til at køre vores container som non-root, derefter har vi det directory hvor det hele foregår i og derefter de porte som der bliver lyttet og hvor vi kan tilgå API'en,

derefter kører vi stage 2 hvor vi bruger et SDK image som kan bygge og det gør vi via ARG med vores build config som er release, som så bliver lagt i vores WORKDIR som hedder /project og derefter kopiere vi vores backend og så render vi en dotnet restore for at hente alle nuget packages, og kopiere vi så resten af vores projekt ned i vores WORKDIR i mappen project/API og til sidst bygger vi vores app, så kommer vi til stage 3 hvor vi gerne vil publish det vi har kørt i gennem i de 2 forrigere stages og så kommer vi til stage 4 hvor vi starter fra vores runtime image og kopiere kun det som vi har published og så starter vi vores app når containeren er oppe og kører. /

Twingate (Freja)

Som en del af kravene til projektet, gør vi brug af Twingate. Twingate er en platform, der tilbyder Zero Trust Networking, og tillader eller nægter adgang til individuelle resurser inden for et privat netværk. Disse tilladelser kan ændres for hver resurse og hver bruger. Dette er i modsætning til en mere traditionel VPN-løsning, hvor der gives adgang til hele netværket, hvor interne resurser skal låses af på anden vis.

Python (Kasper)

Vores producer er lavet med Python dette er gjort fordi det er et stabilt program med mange forskellige extension vi kan gøre brug af, det er sat op som en producer så den producer dataen når arduino er sat til, som så videre sendes til rabbitmq i JSON format, som er omdannet fra det serial output der kommer fra arduinoen af.

```python
import serial
import pika
import json
from datetime import datetime, UTC

# Serial connection (sensor)
ser = serial.Serial('COM5', 9600)

# RabbitMQ connection
connection = pika.BlockingConnection(
    pika.ConnectionParameters('10.133.51.104')
)
channel = connection.channel()
channel.queue_declare(queue='sensor')

print("Producer started...")

while True:
    line = ser.readline().decode().strip()

    try:
        value = int(line)
        timestamp = datetime.now(UTC)

        message = json.dumps({
            "timestamp": timestamp.isoformat(),
            "value": value
        })

        channel.basic_publish(
            exchange='',
            routing_key='sensor',
            body=message
        )

        print("Sent:", message)

    except Exception as e:
        print("Error:", e)
```

/ Billedet oven over viser Python scriptet som starter med at importere serial som bruges til at kommunikere med vores Arduino, derefter importere vi pika som er RabbitMQs bibliotek til at vi kan arbejde med RabbitMQ og efter det importere vi så JSON så vi kan formatere vores serial om til JSON strenge og så importere vi også datetime UTC for at vi benytter os af den valgte datastruktur altså UTC til at sende tiden som kan inputtes i grafen, derefter starter koden med at vi har sat den op til at køre på serial porten “COM5” hvor vi kører med en standard baudrate på “9600” som så læser vores data fra enheden, hvor vi så senere opretter en forbindelse til vores IP via pika.connection hvor vi så efter erklærer at vi vil køre på den kø kaldt sensor, så starter vi vores “producer” som vi får til at kører nonstop så den sender dataen når der er data, derefter tager vi den serielle data og dekoder den og derefter prøver vi at behandle dataen, hvor vi starter med value og konvertere teksten til sensordata, timestamp generere så et aktuelt tidsstempel i UTC format, så laver vi det om til en JSON meddelse med vores tid og værdi, så sender vi den til vores sensor-kø på RabbitMQen så printer vi meddelsen ud for at sikre os at det bliver sendt som vi vil have det og så er der fejlhåndtering nederst og denne fejl vil så blive udskrevet i terminalen hvis der opstår nogen. /

Psycopg2 (Kasper)

Psycopg2 er brugt fremfor Psycopg3 fordi at version 2 af den er fordi den er mere robust og har vist at den fungerer og har stået “the test of time” Psycopg2 gør at python lettere kan implementere mange af de features PostgresSQL tilbyder, dette gøres ved at vi bruger vores Host, db, user og kode til at connecte til den så vi har credentials på at måtte køre dataen videre fra rabbitMQ.

```
# PostgreSQL connection
conn = psycopg2.connect(
    host="10.133.51.104",
    database="sensor",
    user="kasper",
    password="H5kode"
)
cur = conn.cursor()
```

/ Billedet oven over benytter vi os biblioteket kaldet Psycopg2 og derfra er det det ret simpelt vi forbereder vores DB til at kunne udføre SQL-kommandoer og så har vi sådan set bare sat vores credentials ind til at etablere en stabil connection og så kører vi en cursor til for at kunne udføre SQL-queries. /

Pika (Kasper)

Pika er brugt synkront med Psycopg2 da den netop gør brug af de features som den lukker op til at kunne sende AMQP beskeder ud fra vores RabbitMQ i det der hedder et IOloop som gør at den hele tiden modtager og sender på den samme thread.

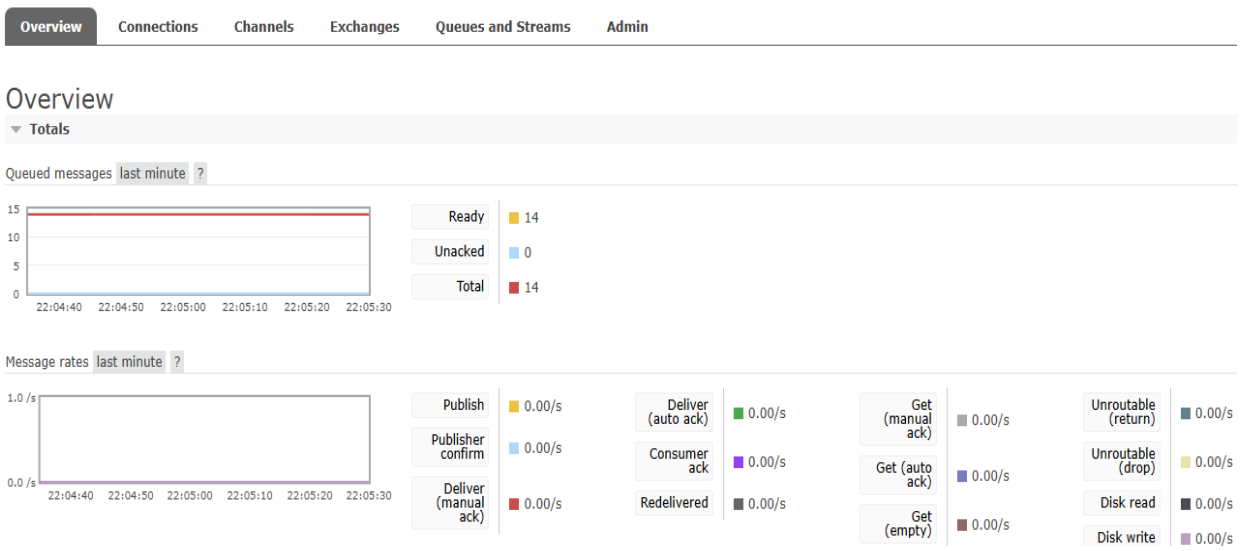
RabbitMQ (Kasper)

RabbitMQ er valgt for at prøve noget andet en HIVEmq, selvom HIVE er mere brugt til iot løsninger, den er sat op til at være en consumer som simpelthen ligger og lytter på om der er noget der kommer ind på queuen der hedder “sensor” som så sender dataen

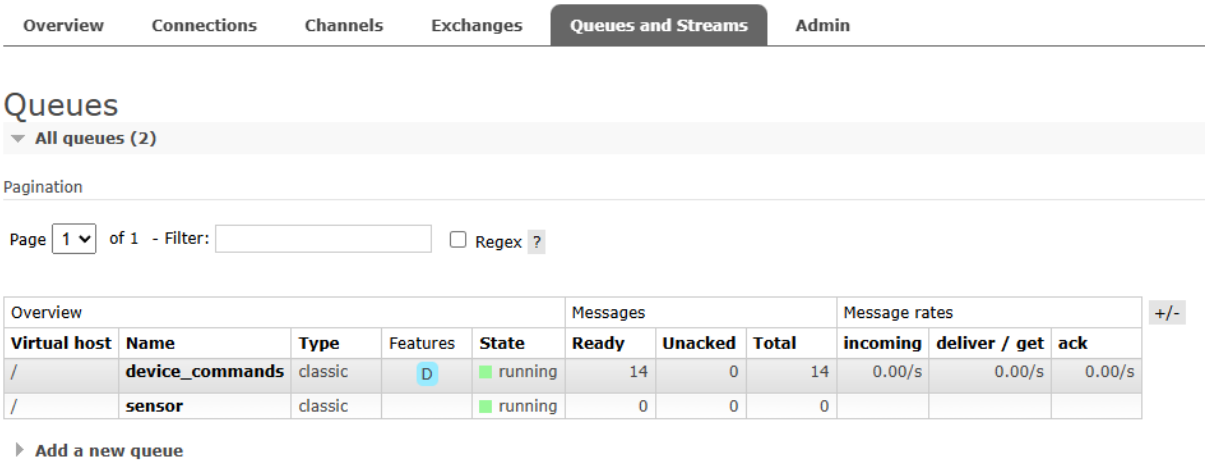
videre til PostgreSQL databasen.

```python
# RabbitMQ connection
connection = pika.BlockingConnection(
    pika.ConnectionParameters('10.133.51.104')
)
channel = connection.channel()
channel.queue_declare(queue='sensor')

print("Consumer waiting for messages...")

def callback(ch, method, properties, body):
    try:
        data = json.loads(body)

        timestamp = data["timestamp"]
        value = data["value"]

        cur.execute(
            "INSERT INTO measurements (timestamp, value) VALUES (%s, %s)",
            (timestamp, value)
        )
        conn.commit()

        print("Saved to DB:", data)

        # Acknowledge message (IMPORTANT)
        ch.basic_ack(delivery_tag=method.delivery_tag)

    except Exception as e:
        print("Error:", e)

        conn.rollback()  # rollback in case of error

# Listen for messages
channel.basic_consume(
    queue='sensor',
    on_message_callback=callback,
    auto_ack=False  # important!
)

channel.start_consuming()
```

/ Billedet oven over viser vores RabbitMQ connection hvor vi starter med at oprette forbindelse til vores server hvor vores RabbitMQ ligger så opretter vi en kanal og der vil vi gerne ramme “sensor” så er der en startbesked hvor den skriver at den venter på meddelser fra producer, så kører vi en def callback som kaldes når en meddelse modtages, derefter henter vi dataen fra vores producer og pakker ud så kører vi en INSERT-SQL for at gemme dataen i tabellen på vores DB i measurements, så commiter vi den modtagne data og så har vi et print på når den går igennem og så har vi en bekræftelse på at meddelsen er modtaget så den kan blive slettet fra RabbitMQ når den er “confirmed” og så har vi igen noget fejl håndtering på hvis noget skulle gå galt, og til sidst har vi en consume på som lytter efter om vi modtager beskeder og så vil kører def callback og den er også hele tiden oppe og køre når den er startet. /



/ Billedet oven over viser det overview/ dashboard man kan tilgå gennem den RabbitMQ vi har sat op på vores server, her kan vi danne os et rimelig godt overblik over hvad der foregår på den hvor vi kan se hvor mange strings der er klar til at blive sendt og hvor vi får en graf på hvor mange der ligger der hvor man på billedet kan se at grafen ligger lige under de 15 på 14 så det passer, også hvor hurtigt vi modtager messages fra vores consumer og vi kan også se hvor hurtigt vi publisher/ sender til vores db hvor mange der bliver confirmed og hvis der sker fejl. /



/ Billedet oven over viser de queues som vi prøver at ramme fra vores consumer script, hvor man på billedet kan se at i den queue der hedder device_commands lige nu er 14 strings som er klar til at blive sendt afsted mod postgre databasen, hvor den queue der hedder sensor allerede, er blevet sendt til databasen og den derfor er på 0 i queuen. /

RestAPI (Silas)

API'en fungerer som object-relational mapping og har ansvaret for at skrive tabeller i vores postgres database samt primært at muliggøre CRUD operations fra Client-side.

API står også for at håndtere login logik ved at validere brugerens loginoplysninger og

udstede en signeret JWT-token som bevis på brugerens identitet. Token fungerer som et adgangskort der automatisk verificerer brugeren ved anmodninger.

Testing environment:
API'er sat op i en samlet .net løsning hvor der udover modeller i et class library er opsat forskellige Unit og integrations tests. Til dette benyttes xUnit og Moq mocking framework.

- Database connection test:
  Tester om vores Postgres database er online og om vores credentials virker.
- End to end integrationstest mod databasen:
  Ved at generere 8000 målepunkter stresstestes systemets evne til at håndtere større datamængder. Samtidig oprettes realistisk testdata i databasen som sikre at 'history' og 'since' queries i API'en har et datasæt for client-side at arbejde med.
- Raw database stress test:
  I denne testes Postgress databasen til sit yderste ved at indsætte 100.000 rækker på samme tid. Testen er sat op mod en dedikeret database i egen container, så den ikke influerer med det øvrige produktionsmiljø.

Dokumentation for API'en findes i h52o_MAIN under Dokumentation -> README.md, UML_KLASSEDIAGRAM.md og DOMÆNEMODEL.md

h52o API

Dette projekt har flere backend-løsninger (micro services) til opsamling og monitorering af solenergidata fra IoT-enheder til fremvisning på Client-side. Systemet er bygget med fokus på sikkerhed, skalerbarhed og realtids-kommunikation. API'ens primære funktioner er ...

Repository og Download

For at hente projektet lokalt kan følgende kommando bruges:

```bash

git clone https://github.com/Mercantec-GHC/h5-projekt-h5-gruppe-2/tree/0a9e45640536aeedd835a7cba70b02d1622609ed/h52o_MAIN

```

Dependencies

API'en anvender følgende biblioteker (NuGet), som findes i projektfilerne.

1. Microsoft.EntityFrameworkCore (10.0.5)
- Primær, gør det muligt at arbejde med databaser som C#-objekter i stedet for rå SQL.

2. Microsoft.EntityFrameworkCore.Design (10.0.5)
- Ansvarlig for migrations samt opdatering databasen.

3. Microsoft.EntityFrameworkCore.Tools (10.0.5)
- Muliggøre kommandoer såsom dotnet ef migrations add mv.

4. Microsoft.VisualStudio.Web.CodeGeneration.Design (10.0.2)

- Hjælpeværktøj til at autogenerere controllers gennem scafolding i VSCode.

5. Npgsql.EntityFrameworkCore.PostgreSQL (10.0.1)
- Gør det muligt for EF Core at kommunikere specifikt med PostgreSQL databasen.

6. Swashbuckle.AspNetCore (10.1.7)
- Bygger Swagger-side med visuel oversigt over endpoints.

7. Microsoft.AspNetCore.Authentication.JwtBearer (10.0.5)
- Giver API'en mulighed for at læse og validere JWT-tokens.

8. BCrypt.Net-Next (4.1.0)
- Sikkerhedsbibliotek der hasher adgangskoder.

9. Microsoft.AspNetCore.OpenApi (10.0.5)
- Hjælpepakke til endpoints efter officiel "OpenAPI" standard, som Swagger bruger.

10. RabbitMQ.Client (7.2.1)
- Gør det muligt for API'en at sende og modtage beskeder via RabbitMQ.

Arkitektur

Systemet er opdelt i en lagdelt struktur:

1. Models og DTOs: Sikrer separation mellem databasestruktur og API-kontrakter for at forhindre cykliske JSON-referencer (Seperation of Concern)

2. Repositories: Håndterer data-adgang med optimerede LINQ-queries (Indsæt note!!) og brug af AsNoTracking (Indsæt note!!) for hurtig læsning.

3. Services: JWT Token-generering og RabbitMQ Message Producer til hardware-kommandoer.

For klassediagram se under UML_KLASSEDIAGRAM.md

Sikkerhed og Autentificering

Systemet benytter følgende sikkerhedsstrategi:

1. BCrypt: Ved oprettelse og login hashes adgangskoder med BCrypt. Dette beskytter mod brute-force angreb og lækage af rå data. (64 bit vs 256!!)

2. JWT: Efter verificeret login modtager klienten et signeret token, der giver adgang til beskyttede endpoints i controllers via [Authorize].

Konfiguration

For at systemet kan køre, skal følgende værdier konfigureres i appsettings.json:

1. ConnectionStrings: Skal indeholde stien tilPostgreSQL databasen (DefaultConnection).

2. Jwt: Skal indholde Key (minimum 64 tegn), Issuer og Audience.

3. RabbitMQ: Skal indeholde Host (Pass?).

IoT Hardware Integration

API'et kommunikerer med hardwaren via RabbitMQ:

1. Start-kommando: Sender en besked til køen device_commands, som instruerer enheden i at starte måling.

2. Stop-kommando: Stopper dataindsamlingen og markerer sessionen som afsluttet i databasen.

Test-vejledning

Vi bruger xUnit til test og har kategoriseret dem for at adskille hurtig feedback fra tung performance-test.

Kør normale tests (Unit og Integration)

Disse tests validerer forretningslogik og database-integration mod den primære database.

```bash

dotnet test --filter "Category!=Performance"

```

Kør performance/stresstest på isoleret database

Denne test udfører en rå SQL-stresstest mod en dedikeret PostgreSQL-container (Port 5433). Formålet er at benchmarke hardwarens maksimale throughput og simulere den belastning, som Python-consumeren påfører systemet, uden overhead fra API eller EF Core.

```bash

dotnet test --filter "Category=Performance" --logger "console;verbosity=detailed"

```

Swagger

Når API'et kører lokalt, kan swagger tilgås her:

http://localhost:5193/swagger/index.html

Swagger kan ligeledes på linux serveren 24/7:

http://10.133.51.104:8081/swagger/index.html

Overblik (Freja)

Vores back end består hovedsageligt af vores Linux-server, Postgres-database, Entity Framework Core API og Twingate-netværk. Derudover gøres der brug af EFCore og RabbitMQ til håndtering af data.

Server og adgang (Freja)

Bagerst har vi vores Linux-server, som kører Ubuntu 24.04. Adgang til den er begrænset til medlemmer i vores Twingate netværk. Twingate står i dette projekt for godkendelse af brugere, og giver direkte forbindelse til serveren, som er inkluderet som en Resource i netværket. Vores Connector er sat op på serveren via en deployet Docker-container, og står for, at vi kan forbinde til den gennem vores private netværk. Vores andre dele/komponenter er også sat op via Docker containere, så alting er samlet og kan eventuelt skaleres i fremtiden, hvis der er brug for det.

Serveren er sat op med 3 forskellige brugere, en til hvert medlem, og de er alle med i en “developers”-gruppe. Gruppen giver adgang til fællesmappen /home/dev, som gør det nemmere at samle fælles filer på serveren.

Brugerne har adgang til sudo i tilfælde af, at der er brug for administratorprivilegier. Dette har vi bestemt, for at hvis det nu skulle ske, at den “hovedansvarlige” for serveren ikke er tilgængelig mens der er brug for noget administrativt, så kan det stadig gøres.

Brugerne har også hver en tilknyttet SSH-nøgle, som gør det muligt at logge på sin egen bruger uden adgangskode via sin personlige computer. Da nøglen er gemt på selve computeren, skaber det ikke ekstra sikkerhedshuller (så længe, at maskinen ikke bruges af uvedkommende), og gør det nemmere og hurtigere for os at tilgå serveren.

Data og håndtering (Kasper)

Dataen produceres gennem fra en phototransistor der måler belysningen og går fra 0-1023 da det er standard for den sensorpin, den data sendes så over serial input til vores Python script som så reader og decoder det og formattere det om til JSON format dette gøres hvert 15ende sekund så når der kommer en ny måling så den sendes videre til vores rabbitMQ på den routing key der hedder "sensor" og den er så sat op til hele tiden at lytte på om der kommer noget ind på den specifikke queue som så sender vores string videre som er "id, timestamp, value" som så bliver sendt ind i vores Postgre DB og opbevares der til at vi så kan hente det ud fra vores API hvilket så bliver fremvist på vores graph udfra vores JSON format.

Dataadgang og grænseflader (Silas)

Der anvendes her en API struktur kombineret med en ORM løsning (Object-Relational mapping). Dette lag fungerer som bindeleddet mellem databasen og client-side applikationen. Der benyttes repositories og data-transfer objects.

Auth

- POST /api/Auth/login

Device

- GET /api/Device
- GET /api/Device/{id}
- DELETE /api/Device/{id}
- GET /api/Device/user/{userId}

Measurement

- GET /api/Measurement/user/{userId}/since
- GET /api/Measurement/user/{userId}/history
- GET /api/Measurement/set/{setId}
- GET /api/Measurement/{id}

MeasurementSet

- POST /api/MeasurementSet/start/{deviceId}
- POST /api/MeasurementSet/stop/{setId}
- GET /api/MeasurementSet/user/{userId}
- GET /api/MeasurementSet/{id}

User

- GET /api/User
- GET /api/User/{id}
- DELETE /api/User/{id}
- POST /api/User/register

Eksempler på GET

Measurement

GET /api/Measurement/user/{userId}/since

Parameters

Cancel

Name	Description
userId * required integer($int32) *(path)*	1
since string($date-time) *(query)*	2026-04-12T06:56:15.178Z

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5193/api/Measurement/user/1/since?since=2026-04-12T06%3A56%3A15.178Z' \
  -H 'accept: text/plain'
```

Request URL

```
http://localhost:5193/api/Measurement/user/1/since?since=2026-04-12T06%3A56%3A15.178Z
```

Server response

Code	Details
200	Response body


```
[
  {
    "id": 2001,
    "value": 285,
    "timestamp": "2026-04-12T10:00:00Z",
    "measurementSetId": 3,
    "deviceId": 1
  },
  {
    "id": 2002,
    "value": 319,
    "timestamp": "2026-04-12T10:00:15Z",
    "measurementSetId": 3,
    "deviceId": 1
  },
  {
```


Download

Response headers

```
content-type: application/json; charset=utf-8
date: Wed,15 Apr 2026 06:56:35 GMT
server: Kestrel
transfer-encoding: Identity
```

Responses

Code	Description	Links

Measurement

GET /api/Measurement/user/{userId}/since

GET /api/Measurement/user/{userId}/history

Parameters Cancel

Name	Description
userId * required integer($int32) (path)	2
anchor string($date-time) (query)	2026-04-12T06:56:15.178Z
minutes integer($int32) (query)	3000

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5193/api/Measurement/user/2/history?anchor=2026-04-12T06%3A56%3A15.178Z&minutes=3000' \
  -H 'accept: text/plain'
```

Request URL

http://localhost:5193/api/Measurement/user/2/history?anchor=2026-04-12T06%3A56%3A15.178Z&minutes=3000

Server response

Code	Details
200	Response body


```
[
  {
    "id": 7001,
    "value": 293,
    "timestamp": "2026-04-11T10:00:00Z",
    "measurementSetId": 8,
    "deviceId": 2
  },
  {
    "id": 7002,
    "value": 319,
    "timestamp": "2026-04-11T10:00:15Z",
    "measurementSetId": 8,
    "deviceId": 2
  },
  {
```


Download

Response headers

```
content-type: application/json; charset=utf-8
date: Wed,15 Apr 2026 07:00:48 GMT
```

Front End

Læsevejledning (Freja)

Brug af billeder heri er som supplerende materiale til teksten enten ved siden af eller over billederne.

OBS: Der er opstået uoverensstemmelser mellem front end og back end, som resultat af ændringer i datastruktur i projektets løb. Koden og strukturen her i front end-delen afspejler den oprindelige aftale, mens back end-delen siden har modtaget større justeringer uden at blive kommunikeret. Dette har medført inkongruens mellem dataobjekter, og resulteret i, at klientens fetch ikke fungerer.

Teknologivalg (Freja)

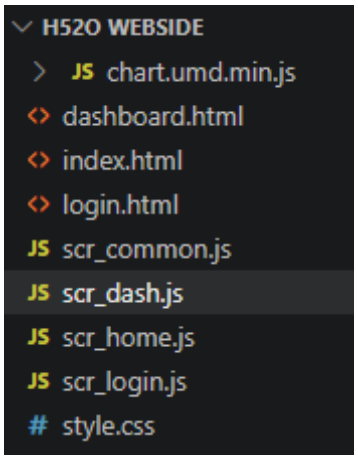
Chart.js (Freja)

Da vi skal visualisere vores data, giver det god mening at gøre det med en graf. Sådan en funktionalitet findes ikke i standard JavaScript eller HTML, så vi gør brug af biblioteket Chart.js (som bliver kaldt bare Chart i resten af afsnittet).

Chart blev valgt, fordi det er meget populært og derfor nemt at finde vejledning og dokumentation for. Det er også nemt at sætte op. Det kræver heller ikke nogen andre udvidelser eller biblioteker for at gøre arbejdet, hvilket har været hjælpsomt i dette projekt, da vi ikke gør brug af andre udvidelser eller frameworks som fx React eller Node. Chart tillader at tilknytte brugerdefinerede akser, farver, animationer, osv, til ønskede graftyper, og spiller nemt med vores datastruktur.

Et alternativ, der blev kigget på, var plot.ly, men det blev ikke valgt på grund af, at det virker tungere end nødvendigt, og har et kæmpe fokus på "AI".

Overblik (Freja)



Vores front end består kun af en samling HTML, CSS og JS. Det dækker de tre forskellige sider, som nuværende eksisterer: index, login og dashboard.

De tre sider har hver deres eget script, men de deler også et til fælles funktionalitet i *scr_common.js*, som fx den topbar, som de alle har. De bruger det samme stylesheet.

Der findes også en lokal kopi af det brugte Chart-bibliotek.

Login (Freja)

Loginsiden har to funktioner: at lave en ny bruger, og verificere en brugers oplysninger. Da der ikke er noget formelt loginsystem på plads endnu, er der brugt en mere midlertidig, "proof of concept"-type opsætning. Det vil sige, at serveren ikke bliver kontaktet her, og at den data, brugeren kan få, ikke har noget med deres egen bruger at gøre.

Når man "opretter" en bruger, gemmes informationerne, som de blev skrevet, i browserens Session Storage. Det vil sige, at de informationer går tabt, når domænet lukkes. Nøglerne/navnene i Session Storage er ikke defineret ift hvad brugeren indtaster, så der kan kun være én "registreret bruger" ad gangen, og fremtidige registreringer overskriver den tidligere, hvis den findes.

```
function RegisterLoginCreds() {
    const regUN = document.getElementById("regUN");
    const regPW = document.getElementById("regPW");
    const regFirstName = document.getElementById("regNameF");
    const regLastName = document.getElementById("regNameL");

    sessionStorage.setItem("username", regUN.value);
    sessionStorage.setItem("password", regPW.value);

    sessionStorage.setItem("firstname", regFirstName.value);
    sessionStorage.setItem("lastname", regLastName.value);
}
```

Når man så "logger ind", sammenlignes teksten i felterne med det, der findes i Session Storage, og hvis de er ens, sættes en logget ind værdi i Session Storage til 1, og man videreføres til dashboardet. Når værdien er 1, erstattes loginknappen i topbaren med en indtil videre funktionsløs knap, der bærer brugerens navn. Her er det tænkt, at den senere skulle fungere som en brugermenu.

```
function TestLogin() {
    const loginUN = document.getElementById("loginUN");
    const loginPW = document.getElementById("loginPW");

    const un = sessionStorage.getItem("username");
    const pw = sessionStorage.getItem("password");

    if (loginUN.value === un && loginPW.value === pw) {
        window.open("/dashboard.html", "_self");
        sessionStorage.setItem("loggedin", "1");
    }
    else {
        alert("Forkert brugernavn eller adgangskode");
    }
}
```

I det færdige produkt bliver disse ting fuldstændigt erstattet med rigtig verificering på serveren med rigtige brugere og deres egen data.

CSS Animation (Freja)

Når man skifter mellem de to formularer (log ind/registrér), spilles et sæt CSS-animationer. CSS-animationer kan bl.a. bruges til at "animere" elementers properties, som fx gennemsigtighed. Normalt ville den ønskede effekt være let at gøre med et værktøj som JQuery, men i stedet blev det afprøvet med ren CSS (med JS til callbacks). Det var ikke det værd, og CSS' animationssystem er ikke særligt robust. Deres egen dokumentation og eksempel på, hvordan man opnår en "fade"-effekt ligesom denne, virker faktisk ikke korrekt. Et uheldigt faktum er, at animationer holder op med at virke, når et element har sit *display* property sat til *none*. Dette betyder, at tilknyttede animationer ikke stopper korrekt, og sender ikke signaler når en animation hverken gennemføres eller afbrydes.

Den originale idé var at kæde to animationer sammen: Den nuværende formular ville blive gradvist usynlig, og den anden formular derefter blive gradvist mere synlig på samme sted. Dette virker ikke, fordi hvis animationen laver display til none som sit sidste trin, så sendes det callback ved afslutningen aldrig.

Lappeløsningen blev stadig at kæde tingene sammen, men med flere forskellige CSS-klasser, hvor animationerne ikke står for ændring af display, og fjernelse af tekstbrydning i stedet gøres via deres *position* property. De forskellige callbacks står derfor for at uddele de rigtige klasser som sluttrin, og animationerne er blevet til mellemtrin.

```
// Igangsætter skift mellem de to formularer
function SwapForms() {
    if (swapInProgress) {
        return;
    }

    swapInProgress = 1;

    if (!formState) {
        formState = 1;

        if (formLogin.classList.contains("FadeIn")) {
            formLogin.classList.remove("FadeIn");
        }

        formLogin.classList.add("FadeOut");
    }
    else {
        formState = 0;

        if (formRegister.classList.contains("FadeIn")) {
            formRegister.classList.remove("FadeIn");
        }

        formRegister.classList.add("FadeOut");
    }
}
```

```
// Når registreringsformularen færdiggører en animation
function RegisterFadeCallback() {
    if (formRegister.classList.contains("FadeOut")) {
        formRegister.classList.add("Hidden");
        formRegister.classList.remove("FadeOut");

        formLogin.classList.remove("Hidden");
        formLogin.classList.add("FadeIn");
    }
    else if (formRegister.classList.contains("FadeIn")) {
        swapInProgress = 0;
    }
}
```

Denne tilgang kan nok ikke anbefales, og kan nemt oversættes til nogle få linjer JQuery i stedet, hvis man vælger at videreudvikle med det.

Dashboard (Freja)

Vores dashboard består af fem hovedparter: grafen/diagrammet, tabellen, felter til tid og dato, knapper med tidsintervaller, og (fetches på) den underliggende data.

Den data, som dashboardet er interesseret i, indeholder kun to ting: tidspunkt og lysstyrke. Dette modtages som JSON, som gør det nemt at oversætte til en simpel klasse med disse to værdier.

Grafen er fra Chart, og er indstillet til at bruge en substring af tidspunkterne inkluderet i vores data som x-akse, og lysstyrken som y-akse. Den tegnes som et almindeligt kurvediagram på toppen af siden, og de forskellige datapunkter kan ses ved at flytte musemarkøren over dem.

Tabellen er en almindelig HTML-tabel. Den bruger det samme data som diagrammet, og tidspunktet er ligeledes en substring, men yderligere forkortet til at fjerne datoen, da forskellige datoer bliver kæmmet ud af datasættet og indsat som ekstra headere i tabellen. Dette gøres for at gøre det mere overskueligt at læse timer, minutter og sekunder, og de ekstra headere vises med en *sticky* position, som får den senest mødte dato til at hænge ved i toppen af tabellen, som man bladrer.

```
// Hvis første objekt i data, eller anden dato, opret ny sticky header med dato
if (i === 0 || currentDay !== lastDay) {
    const row = document.createElement("tr");
    row.className = "TablePatternBreaker";

    const hDate = document.createElement("th");
    hDate.setAttribute("colspan", "2");
    hDate.className = "TableDateHeader";
    const dateString = newData.Time.substring(0, 10);
    hDate.innerText = dateString;
    row.appendChild(hDate);

    table.appendChild(row);

    if (i === 0) {
        lastDay = currentDay;
    }
}
```

Tid og dato kan både manuelt indtastes via en date picker og to felter til henholdsvist timer og minutter, eller ved at trykke på knappen, der siger “Nu”, som automatisk indsætter den lokale tid. Ved manuel indtastning er felterne til timer og minutter opsat med event handlers, som har til opgave at sørge for, at man ikke kan indtaste ugyldige værdier i dem; dette er ikke for at øge sikkerhed (da det skal gøres på server-niveau), men i stedet for at minimere brugerfejl. En testknap på siden kalder en funktion, som oversætter disse felters indhold til en UTC datetime-string. Formålet med disse

elementer er at være i stand til at specificere til vores database/API, hvornår vi gerne vil have data fra.

Intervallet defineres via rækken af knapper, og deres funktion pt er at erstatte den viste data med de forskellige slags testdata. Formålet her er for at kunne findele den data, der kan hentes fra APIen. Da vi gerne vil undgå at hente fx al data fra de sidste 7 dage (op til 7*24*60*4 = 40320 objekter ved 1 måling per 15 sekunder), tænkte vi, det kunne være en god idé at giver længere mellemrum over længere perioder. Dvs give data for hvert minut ved 6 timer, 5. minut ved 24 timer, osv. Dermed kan vi holde os nede på ~200-300 objekter ad gangen, og gøre det mere forståeligt. Der kommer der så en afvejning mellem overblik og præcision, men begge dele er tilgængelige efter ønske.

Når dashboardet åbnes, sættes tid og dato automatisk til lokal tid, og pt det brugte datasæt til 24 timers testdata.

Vores ikke-testdata kan tilgås via en fetch. Da vores API ikke blev færdig, og ikke behandler den udsendte data korrekt, er det kun en meget basal funktionalitet. Den laver en simpel fetch på vores API, og bygger diagrammet og tabellen på ny med den sendte data.

```
async function TestFetch() {
    const requestOptions = {
        method: "GET",
        redirect: "follow"
    };

    fetch("http://10.133.51.104:8080/api/measurement", requestOptions)
        .then((response) => response.text())
        .then((result) => SetUpTable(JSON.parse(result)))
        .catch((error) => console.error(error));
}
```

Meget af koden i dashboardets script går igen, på trods af at testdata og rigtig data bruger samme struktur, da hvordan de tilgås ikke er helt ens. Længere ude i fremtiden ville koden til testdata blive fjernet.

Selveste dashboardsiden er *responsive* til en vis grad, og når viewporten ændrer størrelse, så følger elementer som vores diagram og sidens kolonner med.

JSON Generator (Freja)

Da brugerfladen er bygget til at vise en større mængde data, er det vigtigt at være i stand til at designe smartere omkring det. I dette tilfælde blev der skrevet et lille program i C#, som producerer en ønsket mængde data med både tidspunkt (i UTC) og lysstyrke. Lysstyrken er mest støj og resultatet af RNG-kald, og giver ikke en helt akkurat repræsentation af, hvordan målt data vil se ud, men kan stadig nemt plottes ind i vores graf og tabel.

```
private void WriteOneHour() {
    const string fileName = "dataOneHour.json";
    const int length = 240;

    string strStart = "07/04/2026 04.00.00Z";

    DateTime startOfDay = DateTime.Parse(strStart);

    Data[] data = new Data[length];
    int index = 0;
    DateTime date = startOfDay;

    for (int i = 0; i < length; i++) {
        data[i] = new Data(index++, Utility.rnd.Next(0, 256), date);
        date = date.AddSeconds(15);
    }

    string json = JsonSerializer.Serialize<Data[]>(data);
    File.WriteAllText(fileName, json);
}
```

Programmet danner forskellige data for hvert tidsinterval, og de skrives i en række JSON-filer. Disse læses pt ikke af klienten, men kan i stedet nemt kopieres ind i *scr_dash.js* til test af både graf og tabel.

Den genererede data opfylder ikke helt konceptet om at findele større mængder data, da det ikke er kontinuerligt. Vi har besluttet, at findelingen skal finde sted på server-niveau via forskellige API endpoints eller variation i en endpoint-parameter, og derfor gøres det ikke her.